Day 66 — Layer Caching Optimisation

EngiDock DevOps Training | Month 3: Docker — Containers From Zero

Topic: Measuring Docker build performance and applying caching optimisations

---

Why This Day Matters

On Day 64 you learned that Docker caches each layer. On Day 65 you applied the most important rule: copy dependency files before application code. Today you go deeper — measuring the performance difference, understanding exactly when the cache is invalidated, and applying every technique that makes builds fast.

In a CI/CD pipeline (Month 4), Docker builds run on every commit. A 3-minute build that reinstalls all dependencies on every push versus a 15-second cached build makes a real difference in developer experience and infrastructure cost. At a team of ten developers pushing ten times a day, that is 50 minutes of CI time saved per developer per day.

The techniques here are not optimisations you apply after the fact — they are habits you build from the start. An unoptimised Dockerfile gets slower as the project grows. An optimised Dockerfile stays fast regardless of how large the codebase becomes.

Section 1 — How Docker Layer Caching Works

The Cache Key

For each layer, Docker computes a cache key based on:

1. The Dockerfile instruction text
2. The content of any files referenced by that instruction (for `COPY` and `ADD`)
3. The parent layer's cache key

If any of these change, the cache for that layer is invalidated — and so is the cache for **every subsequent layer**.

```
Layer 1: FROM python:3.11-slim         → Cache key: [image digest]
Layer 2: ENV PYTHONDONTWRITEBYTECODE=1 → Cache key: [layer1_key + instruction]
Layer 3: WORKDIR /app                   → Cache key: [layer2_key + instruction]
Layer 4: COPY requirements.txt .       → Cache key: [layer3_key + file_content_hash]
Layer 5: RUN pip install ...           → Cache key: [layer4_key + instruction]
Layer 6: COPY . .                      → Cache key: [layer5_key + all_file_hashes]
Layer 7: CMD [...]                     → Cache key: [layer6_key + instruction]
```

If `requirements.txt` changes → Layer 4 cache miss → Layers 5, 6, 7 all rebuild.
If only `app.py` changes → Layers 1–5 remain cached → Only Layers 6, 7 rebuild.

Measuring Cache Effect

```
cd ~/flask-docker   # From Day 65

# Cold build (no cache)
docker build --no-cache -t flask-docker .
# Note the time for each step, especially pip install

# Warm build (nothing changed)
docker build -t flask-docker .
# Almost instant — everything cached

# Change only app code
echo "# timing test" >> app.py
time docker build -t flask-docker .
# Fast — pip install cached, only COPY . . and later rebuild

# Change requirements
echo "certifi" >> requirements.txt
time docker build -t flask-docker .
# Slow — pip install must re-run

# Restore requirements
git checkout requirements.txt 2>/dev/null || sed -i '/^certifi$/d' requirements.txt
```

Section 2 — The Wrong Order: Measuring the Problem

Create a deliberately bad Dockerfile:

```
# Dockerfile.bad
FROM python:3.11-slim

ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app

# WRONG ORDER: copies everything before pip install
COPY . .
RUN pip install --no-cache-dir -r requirements.txt

CMD ["python", "app.py"]
```

```
# Build the bad version (cold)
time docker build --no-cache -t flask-bad -f Dockerfile.bad .
# Note the total time

# Change one line in app.py
echo "# a comment" >> app.py

# Rebuild after code change
time docker build -t flask-bad -f Dockerfile.bad .
# SLOW: pip install re-runs because COPY . . was invalidated
```

Now the correct version:

```
# Dockerfile.good
FROM python:3.11-slim

ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app

# CORRECT ORDER: requirements before code
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .

CMD ["python", "app.py"]
```

```
# Change app.py again
echo "# another comment" >> app.py

# Rebuild correct version
time docker build -t flask-good -f Dockerfile.good .
# FAST: pip install cached, only COPY . . runs
```

The difference on a real project with dozens of dependencies: cold pip install = 60–120 seconds. Cached = 0.1 seconds. Every developer push benefits from this.

Section 3 — Optimisation 1: Combine RUN Commands

Each `RUN` instruction creates a layer. Unnecessary layers add overhead and can leave unwanted files (like apt caches) behind in intermediate layers.

The Problem

```
# BAD: 3 layers, apt cache left in layer 1
RUN apt-get update
RUN apt-get install -y curl wget
RUN rm -rf /var/lib/apt/lists/*
```

Each layer is committed to the image independently. The `apt-get update` layer contains the downloaded package lists. Even though the final `rm` layer deletes them, **the data still exists in the intermediate layer** — it is just masked. The image size includes all layer sizes.

The Fix

```
# GOOD: 1 layer, no residual apt cache
RUN apt-get update && \
    apt-get install -y --no-install-recommends \
        curl \
        wget \
        libpq-dev \
    && rm -rf /var/lib/apt/lists/*
```

All three operations in one `RUN` → one layer → the apt cache is never committed to any layer.

The `--no-install-recommends` Flag

```
# Without --no-install-recommends (installs curl + 20 recommended packages)
apt-get install -y curl

# With --no-install-recommends (installs only curl and its direct dependencies)
apt-get install -y --no-install-recommends curl
```

This can save 20–50MB for a typical package.

Section 4 — Optimisation 2: Use .dockerignore Aggressively

Every file in the build context that changes invalidates the `COPY . .` layer. Keep the context clean.

```
# .dockerignore – exclude everything not needed in the image
__pycache__/
*.pyc
*.pyo
*.pyd
.git/
.gitignore
.gitattributes
venv/
.venv/
env/
*.env
.env.*
!.env.example
.pytest_cache/
htmlcov/
.coverage
*.egg-info/
dist/
build/
```

```
node_modules/
*.log
*.sql
*.dump
docs/
tests/
README.md
CHANGELOG.md
docker-compose*.yml
Dockerfile*
.dockerignore
Makefile
```

Measure the Difference

```
# Check context size with .dockerignore
docker build -t myapp . 2>&1 | grep "Sending build context"
# Sending build context to Docker daemon  4.096kB

# Temporarily rename .dockerignore
mv .dockerignore .dockerignore.bak
docker build -t myapp . 2>&1 | grep "Sending build context"
# Sending build context to Docker daemon  12.34MB  (includes venv, .git, etc.)

# Restore
mv .dockerignore.bak .dockerignore
```

Section 5 — Optimisation 3: Multi-Stage Builds Preview

Multi-stage builds are covered in depth tomorrow (Day 67), but here is the concept: use a large build image to compile/install, then copy only the output to a small final image.

```
# Stage 1: Builder — has build tools
FROM python:3.11 AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --user -r requirements.txt

# Stage 2: Final — minimal image
FROM python:3.11-slim AS final
WORKDIR /app
# Copy only the installed packages from builder
COPY --from=builder /root/.local /root/.local
COPY app.py .
ENV PATH=/root/.local/bin:$PATH
CMD ["python", "app.py"]
```

The final image has no pip, no build tools, no package metadata — only the installed packages and your code. You will measure the size difference tomorrow.

Section 6 — Optimisation 4: Pin Base Image by Digest

Tags like `python:3.11-slim` can change when the image maintainer publishes an update. For reproducible builds, pin by digest:

```
# Get the digest of the current image
docker inspect python:3.11-slim --format '{{index .RepoDigests 0}}'
# sha256:abc123def456...

# Use in Dockerfile
FROM python:3.11-slim@sha256:abc123def456...
```

For production CI/CD, pin digests. For day-to-day development, version tags are usually fine.

Section 7 — Optimisation 5: Layer Size Awareness

Know which instructions add size and how much:

```
docker history flask-docker --format "table {{.CreatedBy}}\t{{.Size}}"
```

Typical output for a Flask app:

```
CREATED BY                                      SIZE
CMD ["gunicorn" ...]                            0B
EXPOSE 5000                                     0B
USER appuser                                    0B
COPY . .                                        8kB
RUN pip install ...                             28MB
COPY requirements.txt .                         100B
WORKDIR /app                                    0B
ENV PYTHONDONTWRITEBYTECODE=1 ...               0B
FROM python:3.11-slim                           130MB
```

Observations:

- `FROM python:3.11-slim` = 130MB (the base)
- `RUN pip install` = 28MB (flask, gunicorn, their deps)
- `COPY . .` = 8kB (your actual code — tiny)
- `ENV`, `WORKDIR`, `CMD`, `EXPOSE` = 0B (metadata only)

Total: ~158MB. This is normal for a Python web app.

Section 8 — Optimisation 6: Choosing the Right Base

Base	Size	Use case
`python:3.11`	920MB	Development only — includes compilers, headers, man pages
`python:3.11-slim`	130MB	**Production default** — Debian minimal
`python:3.11-alpine`	50MB	Smallest — but musl instead of glibc (can break packages)
`python:3.11-slim-bookworm`	130MB	Explicit Debian Bookworm version

When to use Alpine:

- Pure Python packages with no C extensions → Alpine works
- Packages with C extensions (psycopg2, Pillow, numpy) → Use slim or install build deps

```
# Test Alpine compatibility
FROM python:3.11-alpine
RUN apk add --no-cache gcc musl-dev postgresql-dev
RUN pip install psycopg2-binary   # May fail on Alpine — use psycopg2 instead
```

For the training, use `python:3.11-slim` for all Python services.

Section 9 — BuildKit: Faster Builds

Docker BuildKit is the modern build engine — enabled by default in Docker 23+. It enables parallel layer building and better caching.

```
# BuildKit is default in modern Docker
docker build -t myapp .

# Explicit (older Docker versions)
DOCKER_BUILDKIT=1 docker build -t myapp .

# With BuildKit, you get better output (progress bars per step)
# and parallel execution of independent stages
```

BuildKit-specific features in Dockerfiles:

```
# syntax=docker/dockerfile:1

# Mount cache for pip (persists across builds, not in the image)
RUN --mount=type=cache,target=/root/.cache/pip \
    pip install -r requirements.txt

# This is faster than --no-cache-dir because pip cache persists between builds
# but is not included in the final image layer
```

```
# Enable experimental syntax
docker build --no-cache -t myapp .
# With --mount=type=cache: pip cache reused even with --no-cache
# pip install takes 2s instead of 30s even on a "cold" build
```

Section 10 — Complete Optimised Dockerfile

Putting all optimisations together:

```
# syntax=docker/dockerfile:1
FROM python:3.11-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1

# Install system dependencies in one layer, clean up in the same RUN
RUN apt-get update && \
    apt-get install -y --no-install-recommends \
        curl \
    && rm -rf /var/lib/apt/lists/*

# Non-root user
RUN addgroup --system appgroup && \
    adduser --system --ingroup appgroup --no-create-home appuser

WORKDIR /app

# Dependencies first (cache-optimised)
COPY requirements.txt .
RUN --mount=type=cache,target=/root/.cache/pip \
    pip install -r requirements.txt

# App code last
COPY --chown=appuser:appgroup . .

USER appuser

EXPOSE 5000
```

```
HEALTHCHECK --interval=30s --timeout=10s --start-period=15s --retries=3 \
    CMD curl -f http://localhost:5000/health || exit 1

CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "4", "app:app"]
```

Key changes from Day 65's version:

- `PIP_NO_CACHE_DIR=1` via ENV (cleaner than per-command flag)
- System deps and cleanup in one `RUN`
- `--mount=type=cache` for pip (BuildKit)
- `COPY --chown` combines COPY and chown in one layer (no extra `RUN chown`)

Section 11 — Measuring: The Full Benchmark

Run this benchmark to see the effect of each optimisation:

```
# Setup
mkdir ~/benchmark && cd ~/benchmark
cat > app.py << 'PYEOF'
from flask import Flask
app = Flask(__name__)
@app.route('/')
def index():
    return 'Hello'
PYEOF

cat > requirements.txt << 'REQEOF'
flask==3.0.0
gunicorn==21.2.0
requests==2.31.0
REQEOF

# Test 1: Bad Dockerfile (wrong order)
cat > Dockerfile.test1 << 'DFEOF'
FROM python:3.11-slim
WORKDIR /app
COPY . .
RUN pip install --no-cache-dir -r requirements.txt
CMD ["python", "app.py"]
DFEOF

time docker build --no-cache -t test1 -f Dockerfile.test1 .
echo "# comment" >> app.py
time docker build -t test1 -f Dockerfile.test1 .
# CODE CHANGE: pip install re-runs (slow)

# Test 2: Good Dockerfile (correct order)
cat > Dockerfile.test2 << 'DFEOF'
FROM python:3.11-slim
WORKDIR /app
```

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "app.py"]
DFEOF

time docker build --no-cache -t test2 -f Dockerfile.test2 .
echo "# another comment" >> app.py
time docker build -t test2 -f Dockerfile.test2 .
# CODE CHANGE: pip install cached (fast)

# Cleanup
docker rmi test1 test2
cd ~
```

Expected results:

- Test 1, second build: ~20-30 seconds (pip runs again)
- Test 2, second build: ~1-2 seconds (pip cached)

Section 12 — Quiz: 15 Questions

Q1. Why does changing `app.py` not invalidate the `pip install` layer in a correctly structured Dockerfile?

A: Because `app.py` is copied by `COPY . .`, which appears *after* the `RUN pip install` layer. Docker builds layers top to bottom. Cache invalidation only flows downward — an invalidated layer invalidates all subsequent layers, but not previous ones. The `pip install` layer's cache key depends on `requirements.txt` (which was copied before it) — since `requirements.txt` did not change, the cache key is unchanged, and the layer is reused.

Q2. Why does putting `RUN apt-get update` and `RUN apt-get install` in separate instructions cause problems?

A: Two issues. First, the apt-get update result (downloaded package lists) is cached in a layer. If you later add a package to the `apt-get install` layer, Docker reuses the cached (possibly outdated) update layer — so the install may fail or install old package versions. Second, the apt cache accumulates in intermediate layers, adding to image size even if a later `RUN rm -rf /var/lib/apt/lists/*` appears to clean it up. Always chain: `apt-get update && apt-get install && rm -rf /var/lib/apt/lists/*` in one `RUN`.

Q3. What does `COPY --chown=user:group` do and why is it better than a separate `RUN chown`?

A: It copies files and sets ownership in a single operation, creating one layer. A separate `RUN chown -R appuser /app` creates a second layer containing a duplicate of all the files (just with different metadata). This doubles the size contribution of those files to the image. `COPY --chown` achieves the same result in one layer with no size penalty.

Q4. What is `.dockerignore` and how does it affect layer caching?

A: `.dockerignore` excludes files from the build context. Files not in the build context cannot change the cache key of `COPY . .`. If your `venv/` (300MB) is in the build context but not in `.dockerignore`, Docker recalculates the hash of the entire directory including `venv/` for every build. This is slow and may cause false cache invalidations if any file in `venv/` changes. With `.dockerignore` excluding `venv/`, Docker only hashes your actual application files — fast and correct.

Q5. What does `--mount=type=cache` do in a BuildKit `RUN` instruction?

A: It mounts a persistent cache directory that is shared across builds but not included in the final image layer. For pip, `--mount=type=cache,target=/root/.cache/pip` means: pip caches downloaded wheels in `/root/.cache/pip`, which persists between builds. So even when the pip install layer is invalidated (requirements.txt changed), pip reuses cached wheels from previous builds — downloads only newly added packages rather than everything. This makes even cache-miss builds significantly faster.

Q6. You have `FROM python:3.11-slim` in your Dockerfile. How do you force a rebuild that uses the very latest version of that image?

A: Two approaches: (1) `docker build --pull -t myapp .` — always tries to pull the latest version of the base image before building. (2) `docker pull python:3.11-slim && docker build --no-cache -t myapp .` — explicitly pull first, then build. Without `--pull`, Docker uses the locally cached version of `python:3.11-slim`, which may be weeks or months old.

Q7. What is the `DOCKER_BUILDKIT=1` environment variable?

A: It enables BuildKit, Docker's modern build engine, for older Docker installations. In Docker 23+, BuildKit is the default and this variable is not needed. BuildKit enables: parallel stage execution in multi-stage builds, better caching, `--mount` directives, and improved output formatting. If you see `--mount=type=cache` in Dockerfiles, BuildKit must be enabled.

Q8. An image is 500MB. You add a 100MB file in one layer and delete it in the next layer. What is the image size?

A: 600MB. The file exists in the layer it was added to — that layer is still part of the image. The delete in the next layer only masks the file (adds a "whiteout" file that hides it), but does not remove the underlying data from the image filesystem. To avoid this, both operations must be in the same `RUN` instruction: `RUN download-file && use-it && rm the-file`.

Q9. How do you see how much each layer contributes to the total image size?

A: `docker history imagename`. The `SIZE` column shows the size added by each layer. Layers from `ENV`, `WORKDIR`, `CMD`, `EXPOSE`, `USER` add 0B — they are metadata. `RUN` and `COPY` instructions typically add the most size. This is how you identify which layers to optimise.

Q10. What is the recommended formula for the number of Gunicorn workers?

A: `(2 × CPU cores) + 1`. On a 1-core container: 3 workers. On a 2-core: 5. On a 4-core: 9. This formula assumes I/O-bound workloads (typical for web apps hitting databases). For CPU-bound workloads, use fewer workers equal to the number of CPU cores. Too many workers wastes memory; too few limits concurrency.

Q11. You change a comment in `app.py`. Does the `COPY . .` layer get invalidated?

A: Yes. Docker computes a content hash of every file in the build context for `COPY . .`. Changing a comment changes the file's content, which changes the hash, which invalidates the layer. This is unavoidable for `COPY . .` — any change to any file in the context invalidates it. This is exactly why you want pip install to be in a layer *before* `COPY . .`, so it remains cached.

Q12. What does `PIP_NO_CACHE_DIR=1` do when set as an ENV?

A: Sets the pip configuration option `no-cache-dir` globally via environment variable. It tells pip never to save downloaded packages to its local cache directory (`~/.cache/pip`). This is equivalent to passing `--no-cache-dir` to every pip command. Since Docker images are immutable and the cache would only add size without benefit, this ENV is a good default. Note: do NOT set this if using `--mount=type=cache` for BuildKit caching — they work in opposite ways.

Q13. How do you rebuild an image completely from scratch, ignoring all cached layers?

A: `docker build --no-cache -t imagename .`. This invalidates all cache and rebuilds every layer from scratch. Use it when: you suspect stale cache is causing issues, you need to pick up security updates in base images, or you want to measure the true cold build time.

Q14. What happens if `requirements.txt` has the same content but a different timestamp?

A: The cache is NOT invalidated. Docker checks content hashes, not file timestamps. If `requirements.txt` has the same bytes (same content), the `COPY requirements.txt .` layer and all subsequent layers remain cached. This is the correct behaviour — timestamps change frequently (e.g., when git restores files) but content changes are what matter.

Q15. You have a 50-package requirements.txt and a 5000-line `app.py`. Which should appear first in the Dockerfile and why?

A: `requirements.txt` should always come first. It changes far less frequently than `app.py` — probably only when you add or update a dependency, perhaps once a week. `app.py` changes on every development session. By putting requirements first, the expensive pip install is cached for the vast majority of builds. If you put `app.py` first, every code change (multiple times per day) forces a full pip reinstall.

Section 13 — Common Mistakes

Mistake 1 — apt-get in separate layers

```
# WRONG — leaves cache in layer 1, cache stale issue in layer 2
RUN apt-get update
RUN apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*

# CORRECT — one layer, no residual cache
RUN apt-get update && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*
```

Mistake 2 — COPY . . before pip install

```
# WRONG — every code change forces full pip reinstall
COPY . .
RUN pip install --no-cache-dir -r requirements.txt

# CORRECT
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
```

Mistake 3 — No .dockerignore

```
# Without .dockerignore: venv/ (200MB), .git/ (50MB), __pycache__/ all included
COPY . .

# With proper .dockerignore: only your application files are copied
```

Mistake 4 — Separate RUN for chown

```
# WRONG — creates extra layer with duplicate file content
COPY . .
RUN chown -R appuser:appgroup /app

# CORRECT — one operation, no extra layer
COPY --chown=appuser:appgroup . .
```

Quick Reference: Build Optimisation Checklist

```
Before pushing any Dockerfile:

□ requirements.txt copied and installed BEFORE COPY . .
□ .dockerignore excludes venv/, .git/, __pycache__/, .env, tests/
□ apt-get update + install + cleanup in ONE RUN
```

```
□ apt-get uses --no-install-recommends
□ pip install uses --no-cache-dir (or --mount=type=cache for BuildKit)
□ COPY --chown instead of COPY + RUN chown
□ Base image is slim/alpine (not full)
□ docker history imagename checked for unexpected large layers

Build timing targets (Flask app with ~10 packages):
- Cold build: 30-60 seconds (acceptable)
- Code-only change rebuild: <5 seconds (target)
- Requirements change rebuild: 30-60 seconds (acceptable)
```

EngiDock DevOps Training | Day 66 of 180
Build real. Break things. Learn why.

Section 14 — Advanced: Cache Busting Techniques

Sometimes you want to force a rebuild of a specific layer without changing your code. This is called cache busting.

Method 1: Build Arguments

```
# Add a build arg whose value you change to bust the cache
ARG CACHEBUST=1
RUN apt-get update && apt-get install -y curl
```

```
# Normal build — uses cache
docker build -t myapp .

# Force apt-get update to re-run by changing CACHEBUST value
docker build --build-arg CACHEBUST=$(date +%s) -t myapp .
```

Method 2: --no-cache on Specific Stage

```
# In multi-stage builds, you can target a specific stage
docker build --target dependencies --no-cache -t myapp .
```

Method 3: Touch a File

```
# Change a file that is copied early in the Dockerfile to bust later layers
touch .cache-bust
# Commit the change — next build invalidates from that COPY instruction forward
```

Section 15 — Layer Cache in CI/CD

In GitHub Actions (Month 4), each CI run starts on a fresh machine with no local Docker cache. Without cache configuration, every build is a cold build.

GitHub Actions provides a Docker layer cache using GitHub's cache storage:

```
# Preview of Month 4 CI configuration
- name: Set up Docker Buildx
  uses: docker/setup-buildx-action@v3

- name: Build and push
  uses: docker/build-push-action@v5
  with:
    context: .
    push: true
    tags: myapp:latest
    cache-from: type=gha        # Read from GitHub Actions cache
    cache-to: type=gha,mode=max # Write to GitHub Actions cache
```

With this configuration, the first CI run on a new branch re-runs pip install. Subsequent runs with no requirements changes: pip install is cached even on CI. This can save 1-3 minutes per pipeline run.

Section 16 — Visualising Your Image with Dive

`dive` is a tool for exploring Docker images layer by layer. It shows you exactly what each layer adds and helps identify waste.

```
# Install dive
wget -q
https://github.com/wagoodman/dive/releases/download/v0.10.0/dive_0.10.0_linux_amd64.deb
sudo apt install -y ./dive_0.10.0_linux_amd64.deb
rm dive_0.10.0_linux_amd64.deb

# Explore your Flask image
dive flask-docker

# Keyboard shortcuts in dive:
# Tab      – switch between layer view and file tree
# Space    – collapse/expand directories
# Ctrl+U   – show only changed/added/removed files
# Ctrl+D   – show only deleted files
# Ctrl+A   – show only added files
```

What to look for with `dive`:

- Large files you did not expect (leftover build artifacts)
- Duplicate files across layers (sign of bad Dockerfile structure)
- Unnecessary files from COPY . . that should be in .dockerignore

---

Section 17 — Summary: The Six Cache Optimisations

#	Technique	Impact	Effort
1	Copy requirements before code	HIGH — pip cached on code changes	Low
2	Combine apt-get in one RUN	MEDIUM — smaller image	Low
3	Use .dockerignore aggressively	HIGH — faster context, correct invalidation	Low
4	Use slim/alpine base image	HIGH — smaller final image	Low
5	COPY --chown instead of RUN chown	LOW — one fewer layer	Low
6	BuildKit --mount=type=cache	HIGH — pip reuses cache even on miss	Medium

Apply optimisations 1–5 to every Dockerfile. Add optimisation 6 for CI/CD pipelines.

Section 18 — Hands-On Lab: Full Optimisation Walkthrough

Follow this complete lab sequence to experience every optimisation.

Lab Setup

```
mkdir ~/caching-lab && cd ~/caching-lab

# Create a realistic Flask app with several dependencies
cat > requirements.txt << 'EOF'
flask==3.0.0
gunicorn==21.2.0
requests==2.31.0
python-dotenv==1.0.0
psycopg2-binary==2.9.9
redis==5.0.1
EOF

cat > app.py << 'EOF'
from flask import Flask, jsonify
import os, time
app = Flask(__name__)
start = time.time()

@app.route('/health')
def health():
    return jsonify({"status": "ok", "uptime": time.time() - start})

@app.route('/')
def index():
    return jsonify({"message": "Caching lab app", "env": os.environ.get("ENV", "dev")})
```

```
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
EOF

# Create a venv to simulate a messy project directory
python3 -m venv venv 2>/dev/null || true

# Add some junk files that should NOT be in the image
mkdir -p tests logs
echo "test content" > tests/test_app.py
echo "log line" > logs/app.log
echo "# dev notes" > README.md
```

Lab Step 1: Measure cold build without .dockerignore

```
# Write a simple Dockerfile
cat > Dockerfile << 'EOF'
FROM python:3.11-slim
WORKDIR /app
COPY . .
RUN pip install --no-cache-dir -r requirements.txt
CMD ["python", "app.py"]
EOF

# Build with no .dockerignore - sends venv/ and everything
time docker build --no-cache -t lab-v1 .
# Note: "Sending build context..." size will be large due to venv/
docker images lab-v1  # Check final image size
```

Lab Step 2: Add .dockerignore

```
cat > .dockerignore << 'EOF'
venv/
__pycache__/
*.pyc
.git/
logs/
tests/
*.md
.env
EOF

# Rebuild — smaller context, same result
time docker build --no-cache -t lab-v2 .
# Context transfer much faster
docker images lab-v2
```

Lab Step 3: Fix instruction order

```
cat > Dockerfile << 'EOF'
FROM python:3.11-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "app.py"]
EOF

# Cold build
time docker build --no-cache -t lab-v3 .

# Change app.py only
echo "# v2" >> app.py

# Incremental build - pip CACHED
time docker build -t lab-v3 .
echo "^^^ Should be <3 seconds"
```

Lab Step 4: Compare image sizes

```
docker images | grep lab-
# lab-v1: probably larger due to venv/ leaking in
# lab-v2, lab-v3: similar, clean
docker history lab-v3
```

Cleanup

```
docker rmi lab-v1 lab-v2 lab-v3
cd ~ && rm -rf ~/caching-lab
```

Section 19 — Real-World Numbers (Reference)

Benchmark results for a typical Flask app with 8 packages (flask, gunicorn, requests, psycopg2, redis, etc.):

Scenario	Build Time
Cold build, wrong order	45–90s
Code change, wrong order	45–90s (pip re-runs)
Cold build, correct order	45–90s
Code change, correct order	1–3s (pip cached)
Requirements change, correct order	45–90s (pip re-runs, expected)

Scenario	Build Time
With BuildKit --mount cache, cold	5–15s (wheels reused)
With BuildKit --mount cache, code change	<1s

The correct instruction order saves ~87 seconds per code-change rebuild. In a busy development session with 20 rebuilds, that is 29 minutes saved per developer per day.

Section 20 — What Is Next

Tomorrow (Day 67) you implement multi-stage builds — the technique that produces the smallest possible production images by separating the build environment from the runtime environment. You will measure before and after: a Python app built with a full Python image vs the same app's runtime in a slim image. The size reduction is typically 50–80%.

EOF
echo "Final: $(wc -c < /sessions/peaceful-exciting-pasteur/mnt/Devops_180days/Docker/Day66_Layer_Caching_Optimisation_Notes.md)"